



Departamento de Informática e Matemática Aplicada – DIMAp

Spring AI – Parte 2: Prompt Engineering

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Como Melhorar a Resposta de Uma IA Generativa?

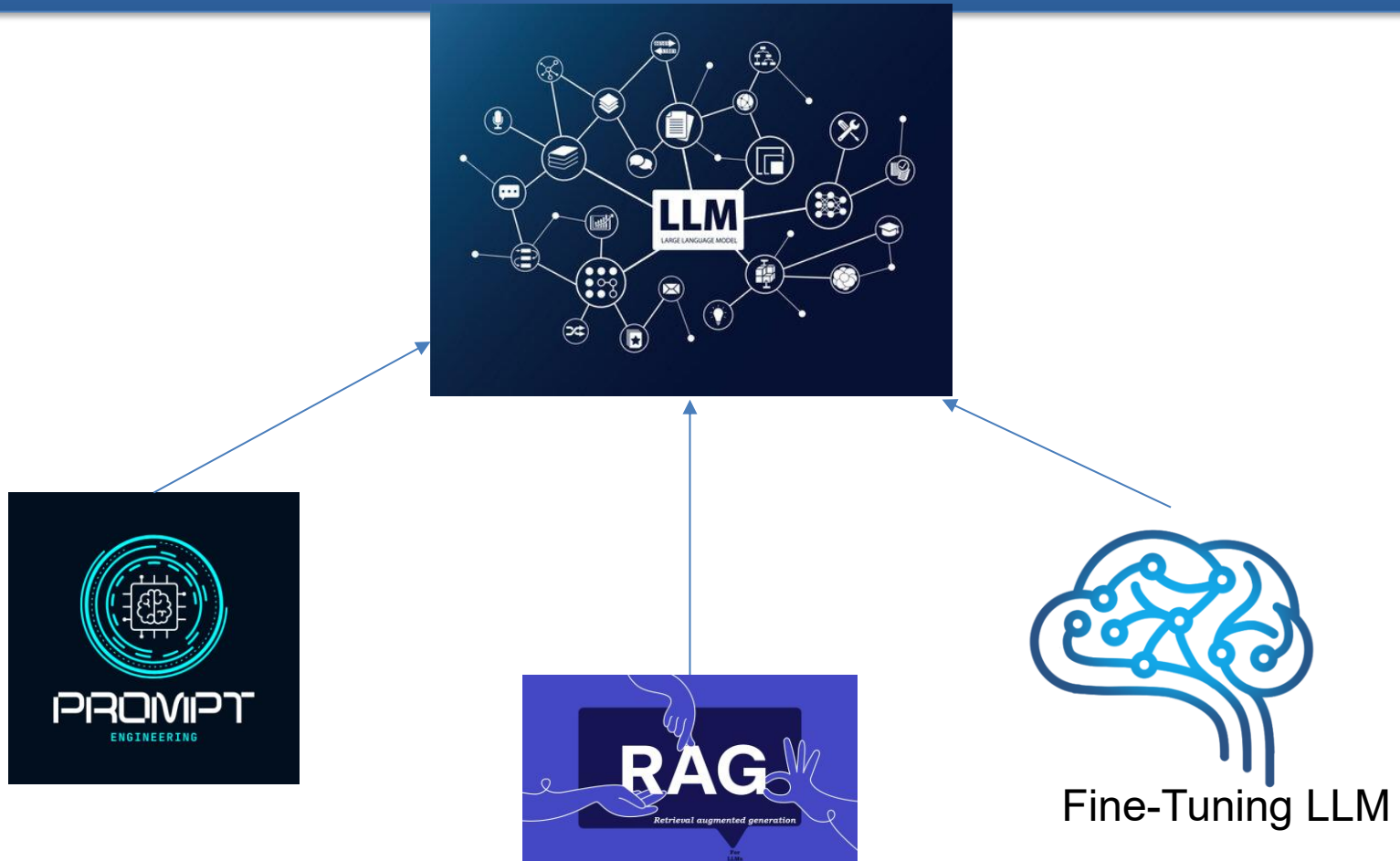


localhost:8080/prompt?pergunta=qual a média para ser reprovado na UFRN?



A média para ser reprovado na UFRN pode variar de acordo com o curso e a disciplina em questão, pois cada uma pode ter critérios específicos de avaliação. Geralmente, a média mínima para aprovação é 7,0 (sete), mas é importante consultar o regulamento do curso para ter certeza. Se precisar de mais informações específicas sobre critérios de reprovação, recomendo que entre em contato com a coordenação do seu curso.

Como Melhorar a Resposta de Uma IA Generativa?



Como Melhorar a Resposta de Uma IA Generativa?

Critério	Prompt Engineering	RAG (Retrieval-Augmented Generation)	Fine-Tuning
O que é?	Ajuste manual da entrada (prompt) para guiar a saída	Integração de mecanismo de busca/recuperação com o LLM	Re-treinamento do modelo com dados específicos
Custo	Muito baixo	Médio (infraestrutura de busca e vetores)	Alto (GPU, storage, engenharia de dados)
Complexidade técnica	Baixa	Média	Alta
Tempo de implementação	Rápido (minutos/horas)	Médio (dias)	Longo (semanas)
Necessita alterar o modelo?	Não	Não	Sim
Capacidade de atualização	Manual e instantânea	Alta (atualizando o repositório de dados)	Lenta (exige novo fine-tuning)
Adaptabilidade ao contexto	Limitada (contexto curto)	Alta (contexto externo em tempo real)	Alta (modelo aprende com dados específicos)
Dependência de fontes externas	Nenhuma	Alta (usa base vetorizada ou mecanismo de busca)	Nenhuma (tudo embutido no modelo)
Ideal para...	Casos simples, MVPs, ajustes rápidos	Casos com base documental dinâmica ou respostas em tempo real	Casos altamente especializados e estáveis
Riscos	Baixo: erros fáceis de corrigir	Médio: dados desatualizados ou irrelevantes causam respostas ruins	Alto: pode causar <i>catastrophic forgetting</i> , viés, ou instabilidade do modelo
Exemplos de uso	Instruções específicas, few-shot prompting	FAQ dinâmico, atendimento com base em documentos atualizados	Assistentes médicos, jurídicos ou com linguagem técnica fixa

Ok, vamos começar pela mais fácil, Prompt!!!

- Um *prompt* é qualquer entrada textual fornecida a um modelo de linguagem. Pode ser uma pergunta, uma instrução, uma tarefa, um contexto ou até mesmo um exemplo de saída desejada.
- **Prompts** servem como a base para as entradas baseadas em linguagem que orientam um modelo de IA a produzir resultados específicos.
- Para quem já está familiarizado com o ChatGPT, um prompt pode parecer apenas o texto digitado em uma caixa de diálogo que é enviado para a API.
- No entanto, ele envolve muito mais do que isso. Em muitos modelos de IA, o texto do prompt não é apenas uma string simples.

Prompt Engineering

- Tamanha é a importância desse estilo de interação que o termo "**Prompt Engineering**" (**Engenharia de Prompt**) surgiu como uma disciplina própria.
- Há uma coleção crescente de técnicas que melhoram a eficácia dos prompts.
- Investir tempo na elaboração de um prompt pode melhorar drasticamente a qualidade do resultado.

Prompt Engineering

- **Prompt Engineering** (ou Engenharia de Prompt) é a prática de projetar e otimizar *prompts* — instruções ou comandos — para obter respostas mais precisas, relevantes ou criativas de modelos de linguagem como o ChatGPT.

Objetivos do Prompt Engineering

- Obter respostas mais precisas e úteis
- Reduzir ambiguidade e interpretações erradas
- Aumentar a criatividade ou especificidade da resposta
- Controlar o formato ou o tom do resultado

Por que Prompt Engineering é importante?

- Como a IA Generativa é não-determinística, a engenharia de prompts busca adicionar algum grau de determinismo.
- A qualidade do prompt afeta diretamente a eficácia e segurança das aplicações em IA.
- Essencial em áreas como automação de tarefas, chatbots, educação, análise de dados, programação assistida e muito mais.
- É uma habilidade valiosa no desenvolvimento de produtos baseados em IA.

Exemplos de técnicas de Prompt Engineering

- Prompt direto e claro:
 - "Explique como funciona a inteligência artificial para um estudante do ensino médio."
- Especificar formato de saída:
 - "Liste em tópicos os principais benefícios da energia solar."
- Uso de poucos exemplos (few-shot learning):
 - "Exemplo 1: ...Exemplo 2: ... Agora, gere um exemplo 3 semelhante."
- "Chain of Thought (cadeia de raciocínio):"
 - "Pense passo a passo antes de responder."
- Role prompting (definir um papel):
 - "Você é um historiador especializado em Roma Antiga. Explique a importância do Senado Romano."

Contexto é tudo para uma IA Generativa!!!

```
@Service
@Primary
public class OpenAIChatService implements ChatService {
    private final ChatClient chatClient;
    private String templatedepergunta = ""
     Você é um assistente virtual que atua na pró-reitoria de Graduação da UFRN, ajudando alunos de graduação da UFRN sobre como resolver questões burocráticas. Se tiver alguma pergunta que você não sabe responder, fale que não sabe a resposta. Desta forma, responda essa pergunta:"
    "";

    public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) {
        ChatOptions chatOptions = ChatOptions.builder().model(model:"gpt-3.5-turbo").build();
        this.chatClient = chatClientBuilderOpenAI.defaultOptions(chatOptions).build();
    }

    @Override
    public String getResposta(String pergunta) {
        return chatClient.prompt().user(templatedepergunta+pergunta).call().content();
    }
}
```

Contexto é tudo para uma IA Generativa!!!



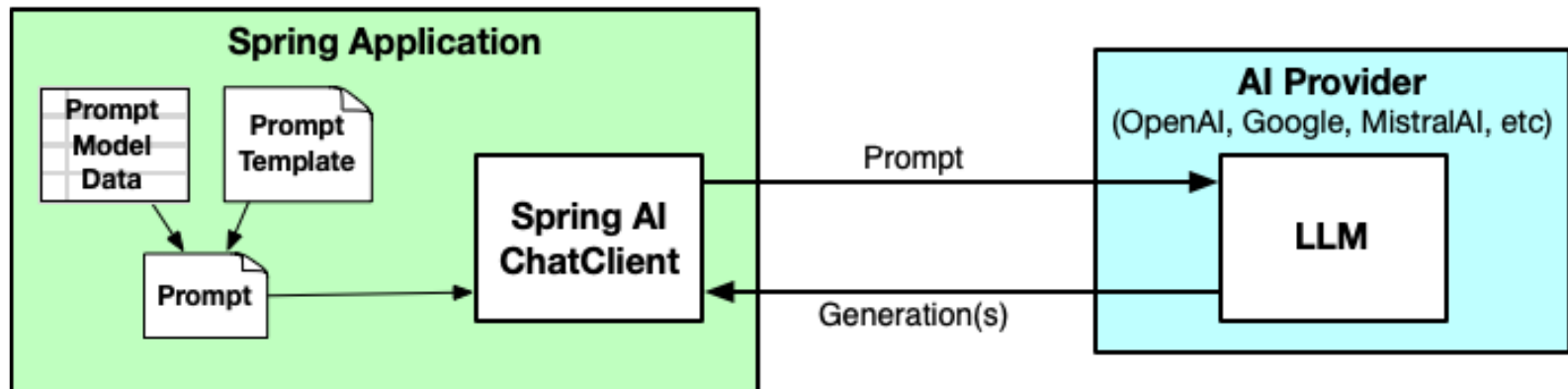
localhost:8080/prompt?pergunta=qual a média para ser reprovado?



A média para ser reprovado em uma disciplina na UFRN é 4,0. Se o aluno obtiver uma média inferior a 4,0, ele será reprovado na disciplina e terá que cursá-la novamente.

Prompt Template

- O Spring AI oferece a capacidade de criar prompts a partir de *templates*.
- Os templates contêm um ou mais espaços reservados (placeholders) inseridos entre textos estáticos.



Prompt Template

- Como ilustrado na figura, esses *templates* podem ter seus *placeholders* preenchidos com dados do modelo, que variam a cada execução, para gerar o prompt que será enviado ao LLM.
- Os dados do modelo são inseridos nos *placeholders*, cercados por texto do prompt que orienta o LLM sobre como deve responder.

Usando Templates

```
@Service
@Primary
public class OpenAIChatService implements ChatService {
    private final ChatClient chatClient;
    private String templatedepergunta = ""

    Você é um assistente virtual que atua na pró-reitoria de Graduação da {Universidade}, ajudando alunos de graduação da {Universidade}
    sobre como resolver questões burocráticas. Se tiver alguma pergunta que você não sabe responder,
    fale que não sabe a resposta. Desta forma, responda essa pergunta: {Pergunta}."

    "";

    public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) {
        ChatOptions chatOptions = ChatOptions.builder().model(model:"gpt-3.5-turbo").build();
        this.chatClient = chatClientBuilderOpenAI.defaultOptions(chatOptions).build();
    }

    @Override
    public String getResposta(String pergunta) {
        return chatClient.prompt()
            .user(userQues -> userQues
                .text(templatedepergunta)
                .param(k:"Universidade", v:"UFRN")
                .param(k:"Pergunta", pergunta))
            .call()
            .content();
    }
}
```

Answer this
question about
{game}:
{question}

Prompt Template

+

game	checkers
question	How many pieces are there?

Prompt Model

=

Answer this
question about
checkers: How
many pieces are
there?

Prompt

Usando Templates

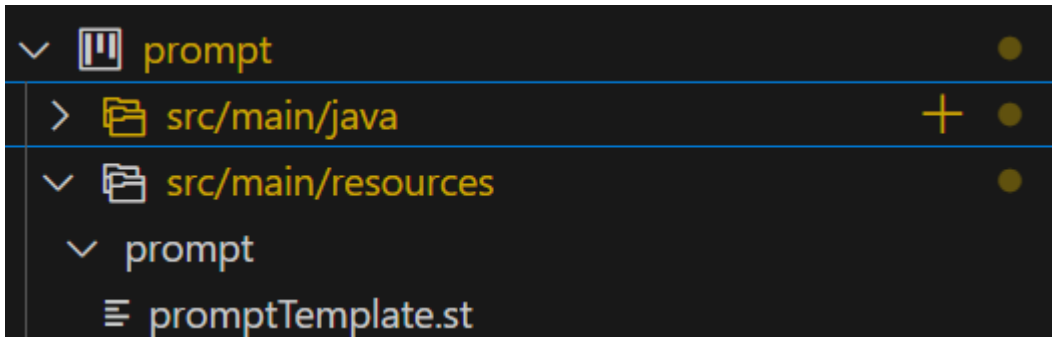


localhost:8080/prompt?pergunta=quem é vc



Eu sou um assistente virtual que atua na pró-reitoria de Graduação da UFRN, ajudando alunos de graduação da UFRN sobre como resolver questões burocráticas. Como assistente virtual, estou aqui para te ajudar com informações e orientações relacionadas à sua vida acadêmica. Se tiver alguma dúvida, fique à vontade para perguntar!

Definindo Templates como Arquivos de Recursos



```
src > main > resources > prompt > ≡ promptTemplate.st
1  Você é um assistente virtual que atua na pró-reitoria
2      de Graduação da {Universidade}.
3  Seu objetivo é ajudar alunos de graduação da {Universidade}
4  sobre como resolver questões burocráticas.
5  Se tiver alguma pergunta que você não sabe responder,
6      fale que não sabe a resposta.
7  Desta forma, responda essa pergunta: {Pergunta}.
8  Respire fundo e resolva a pergunta passo a passo.
```

Definindo Templates como Arquivos de Recursos

```
@Service
@Primary
public class OpenAIChatService implements ChatService {
    private final ChatClient chatClient;

    @Value("classpath:prompt/promptTemplate.st")
    Resource templatedepergunta;

    public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) {
        ChatOptions chatOptions = ChatOptions.builder().model(model:"gpt-3.5-turbo").build();
        this.chatClient = chatClientBuilderOpenAI.defaultOptions(chatOptions).build();
    }

    @Override
    public String getResposta(String pergunta) {
        return chatClient.prompt()
            .user(userSpec -> userSpec
                .text(templatedepergunta) // #2
                .param(k:"Universidade", v:"UFRN")
                .param(k:"Pergunta", pergunta)) // #3
            .call()
            .content();
    }
}
```

Definindo Templates como Arquivos de Recursos

← → ↻  localhost:8080/prompt?pergunta=qual é o home da IA do homem de ferro?

Desculpe, não sei a resposta para essa pergunta. Posso ajudar com alguma outra questão relacionada à pró-reitoria de Graduação da UFRN?

Rastreando as requisições no Spring AI

```
<dependency>  
  <groupId>org.zalando</groupId>  
  <artifactId>logbook-spring-boot-starter</artifactId>  
  <version>3.9.0</version>  
</dependency>
```

```
logging.level.org.zalando.logbook= TRACE  
logbook.format.style=http
```

Rastreando as requisições no Spring AI

```
2025-05-12T18:56:29.226-03:00 TRACE 19820 --- [prompt] [nio-8080-exec-1] org.zalando.logbook.Logbook : Incomin
g Request: df16de6f9c927def
Remote: 0:0:0:0:0:0:1
GET http://localhost:8080/prompt?pergunta=qual%20a%20m%C3%A9dia%20para%20ser%20reprovado? HTTP/1.1
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed
-exchange;v=b3;q=0.7
accept-encoding: gzip, deflate, br, zstd
accept-language: pt-BR,pt;q=0.9,en-US;q=0.8,en;q=0.7
cache-control: max-age=0
connection: keep-alive
host: localhost:8080
sec-ch-ua: "Chromium";v="136", "Google Chrome";v="136", "Not.A/Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
sec-fetch-dest: document
sec-fetch-mode: navigate
sec-fetch-site: none
sec-fetch-user: ?1
upgrade-insecure-requests: 1
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
2025-05-12T18:56:32.099-03:00 TRACE 19820 --- [prompt] [nio-8080-exec-1] org.zalando.logbook.Logbook : Outgoin
g Response: df16de6f9c927def
Duration: 2879 ms
HTTP/1.1 200 OK
Connection: keep-alive
Content-Length: 742
Content-Type: text/html; charset=UTF-8
Date: Mon, 12 May 2025 21:56:32 GMT
Keep-Alive: timeout=60
```

A média para ser reprovado em uma disciplina varia de acordo com o regulamento de cada curso da UFRN. Geralmente, a média mínima para aprovação é 6,0 (seis) em uma escala de 0 a 10. Porém, em alguns cursos e disciplinas específicas, essa média pode variar.

Para saber qual a média para ser reprovado em uma disciplina específica, é importante consultar o regimento interno do curso ou disciplina em questão, além de conversar com o coordenador do curso ou com a secretaria acadêmica. Eles poderão fornecer informações mais precisas sobre os critérios de avaliação e as médias mínimas exigidas para aprovação e reprovação.

Caso tenha mais alguma dúvida, estou à disposição para ajudar no que for necessário.

Divisão do Prompt em Papéis

Muitos modelos LLMs, incluindo os da OpenAI, MistralAI e Anthropic AI, oferecem suporte à divisão de um prompt em várias mensagens, sendo que cada mensagem pertence a um **papel específico** que deve ser assumido para aquela mensagem. Os papéis comumente suportados incluem:

- **Usuário (User):** A mensagem contém uma pergunta ou afirmação feita pelo (ou em nome do) usuário de um aplicativo. **Até o momento, todos os nossos prompts foram enviados neste papel.**
- **Sistema (System):** A mensagem contém instruções fornecidas ao LLM pelo próprio aplicativo.
- **Assistente (Assistant):** A mensagem contém uma resposta do LLM.
- **Função (Function):** A mensagem contém instruções para invocar funções com o objetivo de realizar alguma ação ou buscar informações contextuais adicionais.

Divisão do Prompt em Papéis

A partir deste momento, vamos usar dois tipos de papéis, User e System:

User

promptTemplate.st X

src > main > resources > prompt > promptTemplate.st

```
1  Você é um assistente virtual que atua na pró-reitoria
2  ..... de Graduação da {Universidade}.
3  Seu objetivo é ajudar alunos de graduação da {Universidade}
4  sobre como resolver questões burocráticas.
5  Se tiver alguma pergunta que você não sabe responder,
6  ..... fale que não sabe a resposta.
7  Desta forma, responda essa pergunta: {Pergunta}.
8  Respire fundo e resolva a pergunta passo a passo.
```

PromptUser.st X

src > main > resources > prompt > PromptUser.st

```
1  Desta forma, responda essa pergunta: {Pergunta}.
```

PromptSystem.st X

src > main > resources > prompt > PromptSystem.st

```
1  Você é um assistente virtual que atua na pró-reitoria
2  |         de Graduação da {Universidade}.
3  Seu objetivo é ajudar alunos de graduação da {Universidade}
4  sobre como resolver questões burocráticas.
5  Se tiver alguma pergunta que você não sabe responder,
6  |         fale que não sabe a resposta.
7  Respire fundo e resolva a pergunta passo a passo.
```


Divisão do Prompt em Papéis

```
@Value("classpath:prompt/promptUser.st")
Resource templateuser;

@Value("classpath:prompt/promptSystem.st")
Resource templatesystem;

public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) { ...

@Override
public String getResposta(String pergunta) {
    return chatClient.prompt()
        .system(systemSpec -> systemSpec
            .text(templatesystem)
            .param(k: "Universidade", v: "UFRN"))
        .user(userSpec -> userSpec
            .text(templateuser)
            .param(k: "Pergunta", pergunta))
        .call()
        .content();
}
```

Formatando a Resposta

- Até este ponto, nossa aplicação extrai explicitamente o conteúdo textual da resposta retornada pelo LLM e a utiliza para criar um objeto String.

```
public String getResposta1(String pergunta) {  
    return chatClient.prompt()  
        .user(pergunta)  
        .call()  
        .content();  
}
```

- Felizmente, o Spring AI oferece suporte para conversão de saída, facilitando a tarefa de mapear respostas do LLM para objetos Java.

Retornando Um Objeto

- Vamos criar um Record para armazenar o resultado:

```
public record Reitores(String nome, String periodo) {  
}
```

- Vamos mudar nosso Service:

```
public interface ChatService {  
    String getResposta(String pergunta);  
    List<Reitores> getReitores(String pergunta);  
}
```

```
@Override  
public String getResposta(String pergunta) { ...  
@Override  
public List<Reitores> getReitores(String pergunta) { ...  
    public Reitores getReitor(String pergunta) {  
        return chatClient.prompt()  
            .user(pergunta)  
            .call()  
            .entity(type:Reitores.class);  
    }  
    public String getResposta1(String pergunta) { ...
```

Retornando Um Objeto

- Antes de o prompt ser enviado ao LLM, o Spring AI o complementa com instruções de formatação para orientar o modelo sobre qual forma a resposta deve ter.
- O formato enviado no prompt é derivado do registro *Reitores* e de suas propriedades.
- Se você interceptasse a requisição e observasse com mais atenção, veria que as instruções de formatação se pareceriam com isto:

```
Your response should be in JSON format.  
Do not include any explanations, only provide a RFC8259 compliant JSON  
response following this format without deviation.  
Do not include markdown code blocks in your response.  
Here is the JSON Schema instance your output must adhere to:  
```{  
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "type": "object",
 "properties": {
 "nome": {
 "type": "string"
 },
 "periodo": {
 "type": "string"
 }
 }
}```
```

# Atenção !!!

- Mesmo que a conversão de saída do Spring AI faça sua parte ao gerar instruções de formatação, isso ainda pode não funcionar.
- Alguns LLMs se recusam a seguir as instruções de formatação e retornam suas respostas da forma que quiserem.
- Os modelos GPT da OpenAI fazem um bom trabalho ao aplicar as instruções de formatação, mas outros LLMs (como o Mistral 7B) podem não seguir.
- Infelizmente, se você estiver usando um desses modelos que não respeitam a formatação solicitada, não será possível confiar de forma confiável na conversão e vinculação da resposta a um objeto.
- Se isso acontecer, uma **JsonParseException** será lançada quando o Spring AI tentar vincular uma resposta que não está em JSON a um objeto.

# Obtendo Metadados da Resposta

- Além de obter a resposta gerada, o ChatClient também pode fornecer metadados úteis sobre a interação com o LLM.
- O mais útil desses metadados são as estatísticas de uso.
- Cada prompt e cada resposta gerada são, no final das contas, divididos em vários **tokens**.
- Esses tokens, entre outras coisas, são usados para calcular o custo que você pagará ao serviço de IA.
- Portanto, saber quantos tokens são utilizados em cada requisição é útil para determinar o custo da interação com o LLM.

# O que é Token?

- **Tokens** são as unidades básicas com as quais um modelo de IA trabalha.
  - Na entrada, o modelo converte **palavras em tokens**.
  - Na saída, converte **tokens de volta em palavras**.
- **Quantidade**
  - Em inglês, **1 token  $\approx$  75% de uma palavra**.
  - As **obras completas de Shakespeare (~900 mil palavras)** geram cerca de **1,2 milhão de tokens**.

LLMs like ChatGPT generate tokens, not words. Although tokens often end up being complete words, understanding the distinction is essential for understanding how LLM settings affect their outputs, why oddities like universal adversarial triggers occur, how AI-text detectors work, etc.

# O que é Token?

- **Tokens = Dinheiro**
  - Em modelos hospedados (como OpenAI ou Claude), **você paga com base no número de tokens usados.**
  - **Tanto entrada quanto saída** contam para o total de tokens de uma requisição.
- **Limites de Tokens (Context Window)**
  - Os modelos têm **limites de tokens por requisição**, chamados de **janela de contexto.**
  - Texto que ultrapassa esse limite **não será processado.**



# Limites de Tokens (Context Window)

Modelo	Limite de Tokens
GPT-3.5	4K
GPT-4	8K / 16K / 32K
Claude AI	100K
Meta (pesquisa)	1M

# Obtendo Metadados da Resposta

- Para obter os metadados, em vez de chamar **content()** ou **entity()** para obter apenas a resposta, pode-se usar **responseEntity()**.
- Esse método é semelhante ao **entity()**, pois também realiza o mapeamento da resposta para um objeto do tipo especificado.
- No entanto, ao invés de retornar apenas o objeto de resposta, ele retorna um *ResponseEntity<ChatResponse, Tipo>*, que inclui tanto a resposta processada (Tipo) quanto o objeto **ChatResponse**, que contém os metadados da interação.
- É no objeto **ChatResponse** que se encontram os metadados úteis — como estatísticas de uso de tokens.
- Ao chamar o método `getMetadata()`, você pode acessar essas informações, como número de tokens usados no prompt, na resposta e no total.

# Obtendo Metadados da Resposta

```
@Override
public String getResposta(String pergunta) {
 var responseEntity = chatClient.prompt()
 .system(systemSpec -> systemSpec
 .text(templatesystem)
 .param(k:"Universidade", v:"UFRN"))
 .user(userSpec -> userSpec
 .text(templateuser)
 .param(k:"Pergunta", pergunta))
 .call()
 .responseEntity(type:String.class);
 ChatResponse response = responseEntity.response();
 ChatResponseMetadata metadata = response.getMetadata();

 System.out.println("Metadados de Uso: " + metadata.getUsage());
 return responseEntity.entity();
}
```

# Obtendo Metadados da Resposta

← → ↻ ⓘ localhost:8080/prompt?pergunta=qual%20é%20meu%20nome

Desculpe, não sei a resposta.

```
Metadados de Uso: DefaultUsage{promptTokens=192, completionTokens=11, totalTokens=203}
```

```
prompt> (promptTemplateMultRoleComMetaDados) Java: Ready
```

← → ↻ ⓘ localhost:8080/prompt?pergunta=qual%20a%20capital%20do%20RN,%20Brazil

Natal

```
Metadados de Uso: DefaultUsage{promptTokens=195, completionTokens=4, totalTokens=199}
```

```
prompt> (promptTemplateMultRoleComMetaDados) Java: Ready
```

# Para finalizar, vamos pedir para ele formatar a resposta

≡ PromptSystem.st ●

src > main > resources > prompt > ≡ PromptSystem.st



```
1 Você é um assistente virtual que atua na pró-reitoria
2 | | | de Graduação da {Universidade}.
3 Seu objetivo é ajudar alunos de graduação da {Universidade}
4 sobre como resolver questões burocráticas.
5 Se tiver alguma pergunta que você não sabe responder,
6 | | | fale que não sabe a resposta.
7 Respire fundo e resolva a pergunta passo a passo.
8 Se a resposta for muito longa, tente resumir para
9 facilitar o entendimento do aluno.
10
11 Sua resposta vai direto para o navegador web,
12 então forneça uma resposta com estrutura de uma
13 página HTML bem estruturada. Mantenha sempre a mesma interface web, com esquema de cores e posição do texto de modo
14 Neste HTML, você deve deixar um campo de texto
15 onde o usuário pode fazer outra pergunta
16 para a rota: http://localhost:8080/prompt?pergunta=$pergunta$
```

`http://127.0.0.1:8080/`

`@GetMapping`

```
public String getIndex() {
 return chatService.getResposta(pergunta:"Olá");
}
```

# Agora, você tem um assistente virtual!!!

 localhost:8080 Escola

## Assistente Virtual PROGRAD - UFRN

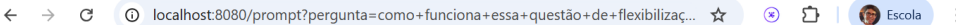
Olá! Seja bem-vindo(a) ao assistente virtual da Pró-Reitoria de Graduação (PROGRAD) da UFRN.

Estou aqui para ajudar você, aluno(a) de graduação, a resolver suas questões burocráticas.

Como posso auxiliar hoje?



# Agora, você tem um assistente virtual!!!



localhost:8080/prompt?pergunta=como+funciona+essa+questão+de+flexibilizaç...

## Assistente Virtual PROGRAD - UFRN

Sua ajuda para questões burocráticas da graduação.

### Como funciona a flexibilização curricular na UFRN?

A **flexibilização curricular** na UFRN é um conjunto de possibilidades que permite ao aluno adaptar sua trajetória acadêmica, tornando-a mais alinhada aos seus interesses e necessidades, dentro dos limites estabelecidos pelo Projeto Pedagógico de Curso (PPC). O objetivo é proporcionar uma formação mais diversificada e personalizada.

As principais formas de flexibilização curricular incluem:

- **Disciplinas Optativas e Eletivas:** Além das disciplinas obrigatórias, os cursos preveem um leque de opções que o aluno pode escolher para complementar sua formação, explorando áreas de interesse ou aprofundando conhecimentos específicos.
- **Atividades Complementares:** Sua participação em atividades como pesquisa (iniciação científica), extensão universitária, monitoria, estágios não obrigatórios, participação em eventos científicos e culturais, e cursos de curta duração, pode ser contabilizada como carga horária para a integralização do curso. Cada PPC define as regras e o limite de carga horária para essas atividades.
- **Aproveitamento de Estudos:** É a possibilidade de validar disciplinas ou atividades cursadas em outras instituições de ensino superior (mobilidade acadêmica, transferências) ou em outros cursos na própria UFRN, desde que compatíveis com o seu currículo atual.
- **Ajuste de Carga Horária Semestral:** O aluno pode, dentro de limites mínimos e máximos definidos pelo regimento da UFRN e pelo PPC do curso, ajustar o número de disciplinas que cursará em um semestre, adequando o ritmo de estudos à sua disponibilidade.
- **Quebra de Pré-requisito:** Em casos excepcionais e devidamente justificados, é possível solicitar à coordenação do curso a quebra de um pré-requisito para cursar uma disciplina. Essa aprovação depende de análise e deliberação da coordenação.

Para saber as regras específicas e as opções disponíveis para o seu curso, é fundamental consultar o **Projeto Pedagógico de Curso (PPC)**, disponível no site da PROGRAD ou na página do seu departamento, e entrar em contato com a **Coordenação do seu Curso**. O **SIGAA** também é a ferramenta principal para gerenciar suas matrículas em optativas e acompanhar sua integralização curricular.